

FORMULARIO SENCILLO (SOLO 2 CAMPOS)

Vamos a trabajar **solo con**:

- nombre
- email

Esto nos permite centrarnos **solo en el envío a la API**, sin ruido.

PASO 1: ¿POR QUÉ USAMOS `useState`?

En React **los inputs NO se leen del HTML**, se leen del **estado**.

React trabaja con el concepto de
“**Single Source of Truth**” (una única fuente de verdad)

Esa fuente es el **estado**.

Por eso usamos `useState`.

EXPLICACIÓN PEDAGÓGICA

¿POR QUÉ usamos `name` en los `inputs`?

Porque el `name` nos dice **qué campo estamos modificando**.

```
name="nombre"
```

```
name="email"
```

Luego React lo usa aquí:

```
[name]: value
```

Sin `name`, no sabríamos qué propiedad del estado actualizar

🔗 ¿QUÉ HACE `handleChange` REALMENTE?

Cada vez que el usuario escribe:

1. React captura el evento
2. Obtiene:
 - el nombre del campo (`name`)
 - el valor escrito (`value`)
3. Actualiza SOLO ese campo en el estado

- ☐ Esto permite **un solo manejador para todos los inputs**

🔗 ¿POR QUÉ usamos `preventDefault()`?

Un `<form>` HTML normal:

- recarga la página
- borra el estado
- rompe React

- ☐ React NO quiere recargas
- ☐ React quiere manejar todo con JavaScript

🔗 ¿QUÉ ES `fetch`?

`fetch` es la forma moderna de **hacer peticiones HTTP**:

- Enviar datos (POST)
- Recibir respuestas (JSON)
- Hablar con un servidor

🔗 ¿POR QUÉ `async` / `await`?

Las peticiones a una API:

- NO son instantáneas
- tardan un tiempo

`async` / `await` permite:

- escribir código claro
- esperar la respuesta
- manejar errores fácilmente

¿QUÉ SE ENVÍA A LA API?

```
body: JSON.stringify(form)
```

Esto envía EXACTAMENTE:

```
{
  nombre: "Juan",
  email: "juan@email.com"
}
```

🔗 ¿QUÉ DEVUELVE JSONPLACEHOLDER?

Devuelve algo así:

```
{  
  
  id: 101,  
  
  nombre: "Juan",  
  
  email: "juan@email.com"  
  
}
```

ENVÍO DE FORMULARIOS A UNA API EN REACT

Explicación pedagógica de fetch, async/await y isSubmitting

1. ¿POR QUÉ NECESITAMOS fetch?

En una aplicación React:

- El formulario recoge datos del usuario
- Pero **React por sí solo no guarda datos en un servidor**
- Para enviar esos datos necesitamos comunicarnos con una **API**

`fetch` es la herramienta que nos permite:

- Enviar datos a un servidor
- Recibir una respuesta
- Saber si todo ha ido bien o ha fallado

En este caso usamos la API de prueba **JSONPlaceholder**.

2. ¿QUÉ ES UNA API?

Una API es un programa que:

- Recibe datos
- Procesa datos
- Devuelve una respuesta

JSONPlaceholder simula una API real para practicar.

La dirección que usamos es:

`https://jsonplaceholder.typicode.com/posts`

3. ¿POR QUÉ USAMOS `POST`?

Existen distintos tipos de peticiones HTTP:

- `GET` → pedir datos
- `POST` → enviar datos
- `PUT` → modificar datos
- `DELETE` → borrar datos

Cuando enviamos un formulario usamos **`POST`**, porque estamos **enviando información nueva**.

4. ESTRUCTURA COMPLETA DEL ENVÍO CON `fetch`

```
const response = await fetch(
  'https://jsonplaceholder.typicode.com/posts',
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(form)
  }
);
```

Ahora lo explicamos por conceptos.

5. ¿QUÉ SIGNIFICA `await fetch`?

Las peticiones a un servidor:

- Tardan un tiempo
- No son inmediatas

`await` significa:

“Espera a que el servidor responda antes de continuar”

Esto evita errores y nos permite trabajar con la respuesta correctamente.

6. ¿QUÉ ES headers?

```
headers: {  
  'Content-Type': 'application/json'  
}
```

Los headers son información adicional que enviamos al servidor.

En este caso le decimos:

“Los datos que te envío están en formato JSON”

Si no lo indicamos, el servidor puede no entender los datos.

7. ¿QUÉ ES body Y POR QUÉ USAMOS JSON.stringify?

El `body` contiene **los datos del formulario**.

Nuestro formulario es un objeto JavaScript:

```
{  
  nombre: 'Juan',  
  email: 'juan@email.com'  
}
```

Pero el servidor **NO entiende objetos JavaScript**.

Por eso usamos:

```
JSON.stringify(form)
```

Esto convierte el objeto en texto JSON, que sí entiende el servidor.

8. ¿QUÉ ES response?

```
const response = await fetch(...)
```

`response` es la respuesta que devuelve el servidor.

Contiene:

- Si la petición fue correcta
- El código HTTP
- Los datos devueltos

9. ¿POR QUÉ USAMOS `response.json()`?

El servidor responde en formato JSON.

```
const data = await response.json();
```

Esto convierte la respuesta JSON en un objeto JavaScript que podemos usar en React.

10. ¿QUÉ ES `try / catch / finally`?

Cuando trabajamos con APIs pueden ocurrir errores:

- Sin conexión a internet
- El servidor falla
- La API no responde

Por eso usamos:

```
try {  
  // intento enviar los datos  
} catch (error) {  
  // algo ha fallado  
} finally {  
  // se ejecuta siempre  
}
```

11. ¿QUÉ ES `isSubmitting` Y PARA QUÉ SIRVE?

```
const [isSubmitting, setIsSubmitting] = useState(false);
```

`isSubmitting` es un estado que indica:

- Si el formulario se está enviando
 - Si debemos bloquear el botón
-

12. ¿POR QUÉ USAMOS `setIsSubmitting(true)`?

Antes de enviar los datos hacemos:

```
setIsSubmitting(true);
```

Esto significa:

- El formulario está en proceso de envío
- El usuario no debe pulsar el botón otra vez

Normalmente se usa para:

- Deshabilitar el botón
- Mostrar “Enviando...”

13. ¿POR QUÉ `setIsSubmitting(false)` VA EN `finally`?

```
finally {  
    setIsSubmitting(false);  
}
```

`finally` se ejecuta **SIEMPRE**, ocurra lo que ocurra.

Esto garantiza que:

- El botón vuelva a activarse
- La interfaz no quede bloqueada
- El usuario pueda volver a intentarlo

Si no usamos `finally`, el botón podría quedarse bloqueado para siempre.

14. ¿CÓMO SABEMOS QUE LOS DATOS SE HAN ENVIADO?

Hay dos formas:

1. Consola del navegador

```
console.log(form);  
console.log(data);
```

2. Herramientas del navegador

- F12
 - Pestaña Network
 - Buscar la petición POST
 - Ver Payload y Response
-

16. RESUMEN FINAL

- `fetch` envía datos a un servidor
 - `POST` se usa para enviar formularios
 - `headers` indican el formato de los datos
 - `JSON.stringify` convierte los datos
 - `await` espera la respuesta
 - `try/catch` controla errores
 - `finally` limpia el estado
 - `isSubmitting` controla la interfaz
-

Código completo del formulario

```
import { useState } from 'react'
import './App.css'

function App() {
  const [form, setForm] = useState({
    nombre: '',
    email: '',
  });
  // 2 Estado para saber si se está enviando
  const [enviadoF, setenviadoF] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target
    setForm({
      ...form,
      [name]: value
    })
  }

  const handleSubmit = async(e) => {
    e.preventDefault()// evita recarga de la página
    setenviadoF(true); // 3 Indicamos que se está enviando
    const api_url =
'https://6970b24178fec16a63fea11f.mockapi.io/usuarios';
    //validaciones de los campos de fl formulario
    const nombreRegex = /^[A-Za-zÁÉÍÓÚáéíóúÑñ\s]+$/;
    if (!nombreRegex.test(form.nombre)) {
      console.log('Error: el nombre solo puede contener letras
y espacios');
      return;
    }else {
      try {
        const respuesta = await fetch(api_url,
          {
            method: 'POST',
            headers: {
              'Content-type': 'application/json; charset=UTF-8',
            },
            body: JSON.stringify(form)
          });
        const datos = await respuesta.json();
        console.log(' Datos enviados:');
        console.log(form);
        console.log(' Respuesta del servidor:');
        console.log(datos);
      } catch (error) {
        console.log('Error al enviar los datos');
      }
    }
  }
}
```

```

    }catch (error) {

        console.log('❌ Error al validar el nombre:', error);

    }finally {

        setenviadoF(false); // 4️⃣Indicamos que ha terminado de enviarse

    }

}
console.log('Nombre válido');

//todo ha ido bien lo envio
console.log(`Nombre enviado: ${form.nombre}`)
console.log(`Email enviado: ${form.email}`)
}

return (
    <>
    <h1>Formulario de Registro</h1>
    <form onSubmit={handleSubmit}>
        <label htmlFor="nombre">Nombre:</label>
        <input
            type="text"
            name="nombre"
            value={form.nombre}
            onChange={handleChange}
            placeholder='Escribe tu nombre'/>
        <input
            type="email"
            name="email"
            value={form.email}
            placeholder='Escribe tu email'
            onChange={handleChange}
        />
        <button type="submit">Enviar</button>
    </form>
    </>
)
}

export default App

```